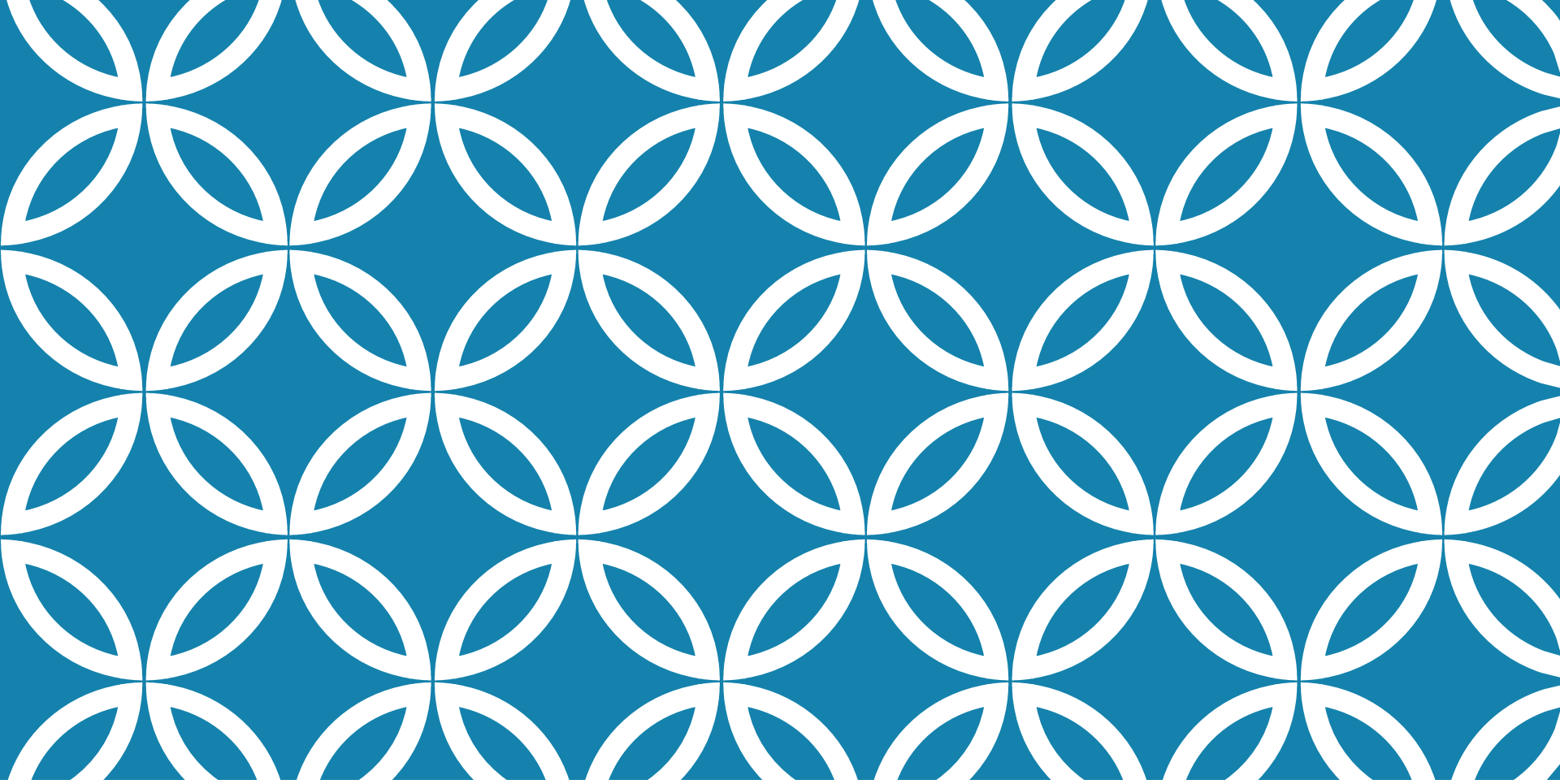


# BACKTRACKING, BRANCH-AND-BOUND APPROXIMATION ALGORITHMS

Assoc.Prof. Suphakant Phimoltares, Ph.D.
Advanced Virtual and Intelligent Computing (AVIC) Center

# AGENDA

1. Backtracking
2. Branch-and-Bound
3. Approximation algorithms

# TACKLING DIFFICULT PROBLEMS

There are two approaches to solve difficult problems

1. Use a strategy that guarantees solving the problem exactly but doesn't guarantee to find a solution in polynomial time
2. Use an approximation algorithm that can find an approximate (sub-optimal) solution in polynomial time

# EXACT SOLUTION STRATEGIES

*exhaustive search* (brute force)

- useful only for small instances

*dynamic programming*

- applicable to some problems (e.g., the knapsack problem)

*backtracking*

- eliminates some unnecessary cases from consideration
- yields solutions in reasonable time for many instances but worst case is still exponential

*branch-and-bound*

- further refines the backtracking idea for optimization problems

# BACKTRACKING

Construct the *state-space tree*

- nodes: partial solutions
- edges: choices in extending partial solutions

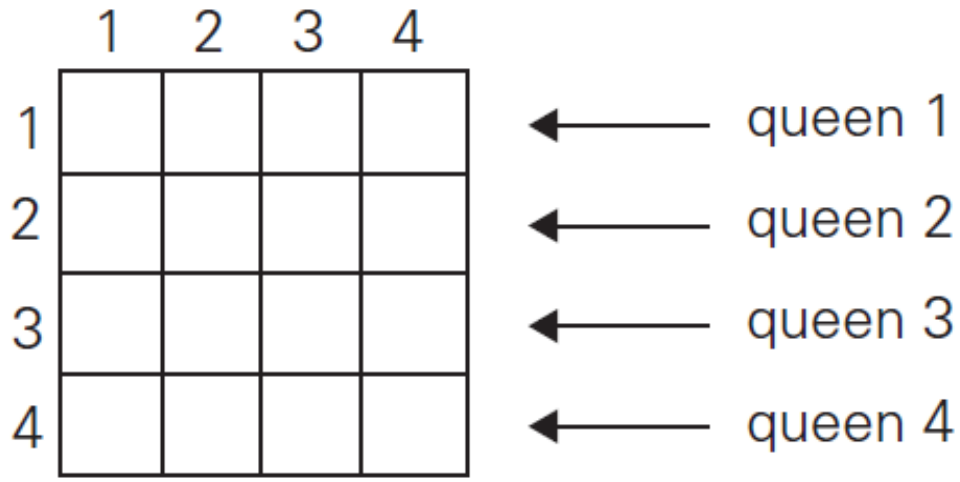
Explore the state-space tree using depth-first search

"Prune" *nonpromising nodes*

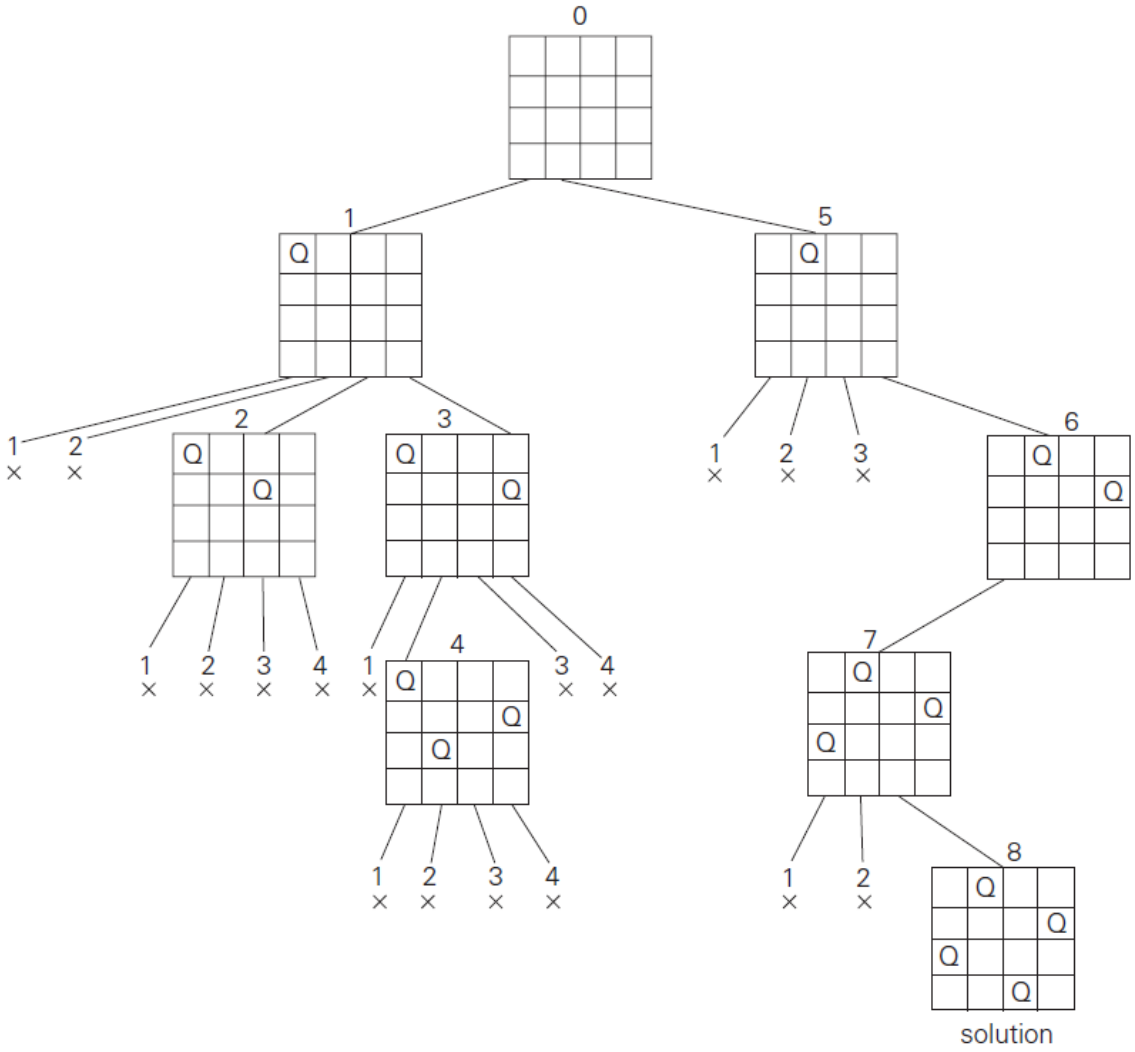
- stop exploring subtrees rooted at nodes that cannot lead to a solution and backtracks to such a node's parent to continue the search

# *N*-QUEENS PROBLEM

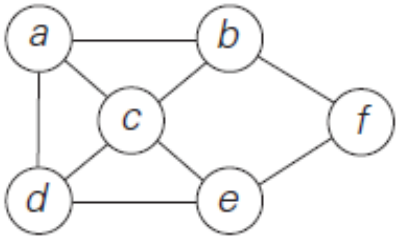
Place *n* queens on an *n*-by-*n* chess board so that no two of them are in the same row, column, or diagonal

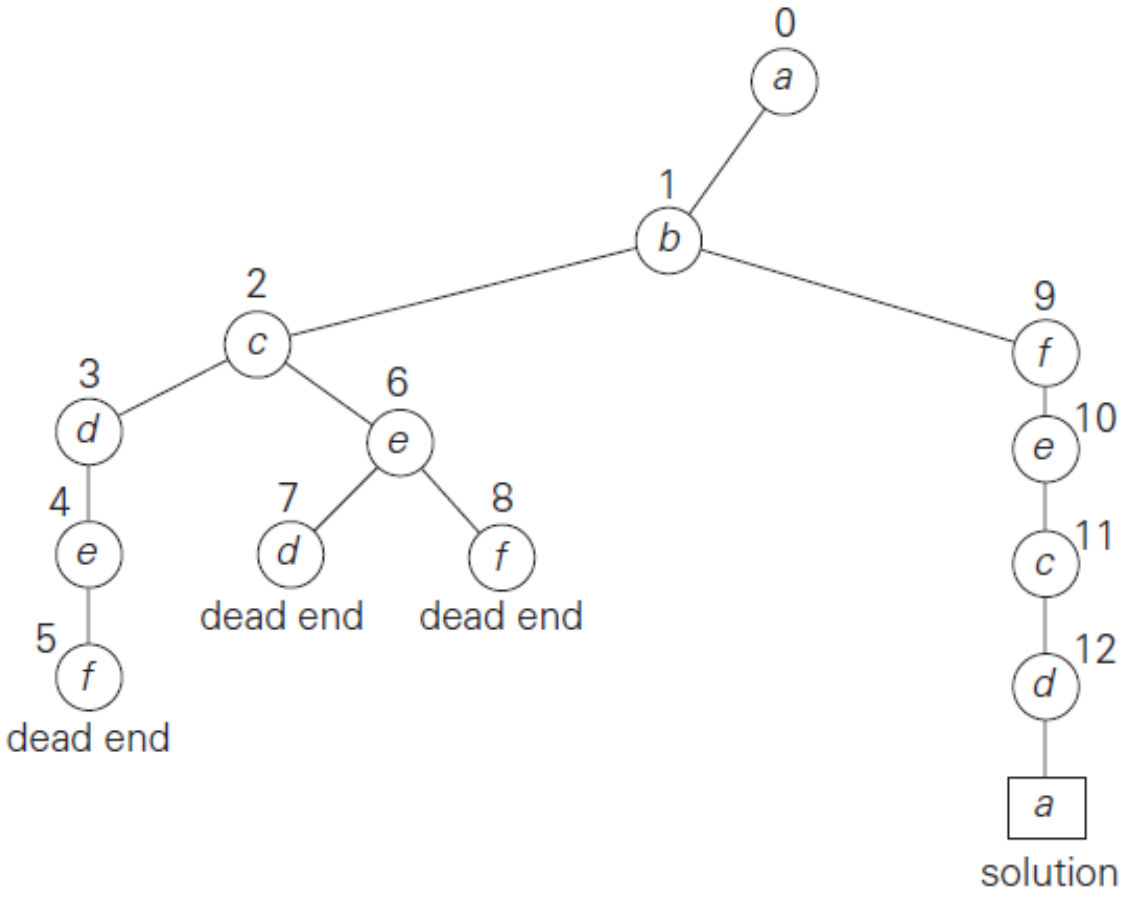


# STATE-SPACE TREE OF THE 4-QUEENS PROBLEM



# HAMILTONIAN CIRCUIT PROBLEM



# BRANCH-AND-BOUND

An enhancement of backtracking

Applicable to optimization problems

For each node (partial solution) of a state-space tree, computes a bound on the value of the objective function for all descendants of the node (extensions of the partial solution)

Uses the bound for:

- ruling out certain nodes as "nonpromising" to prune the tree – if a node's bound is not better than the best solution seen so far
- guiding the search through state-space

# EXAMPLE: ASSIGNMENT PROBLEM

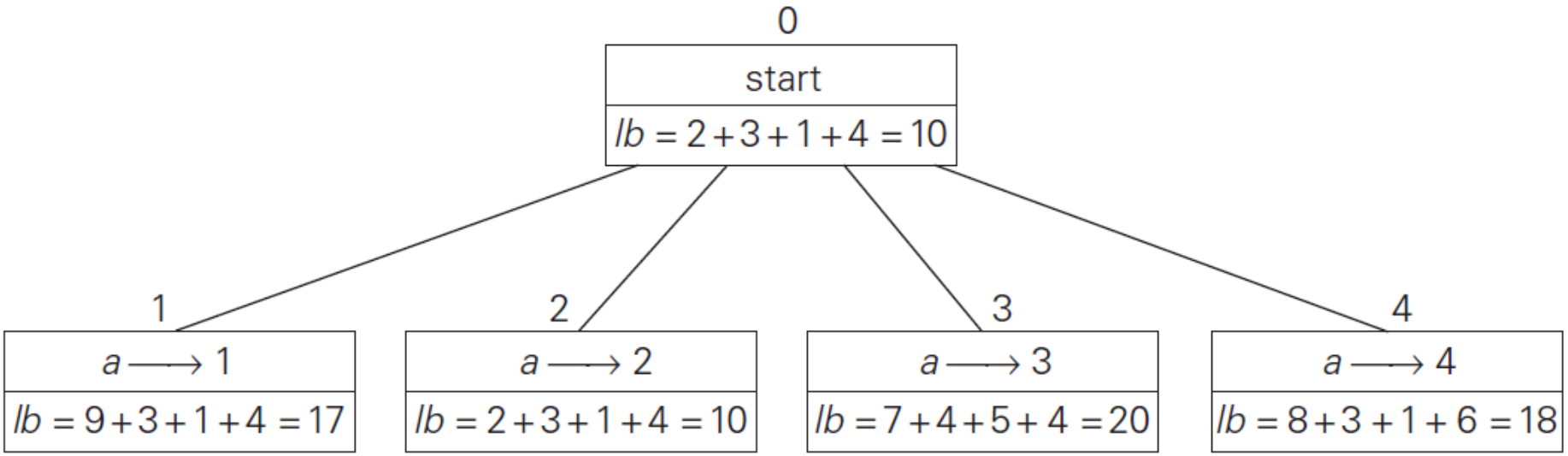
Select one element in each row of the cost matrix C so that:
- no two selected elements are in the same column
- the sum is minimized

Example

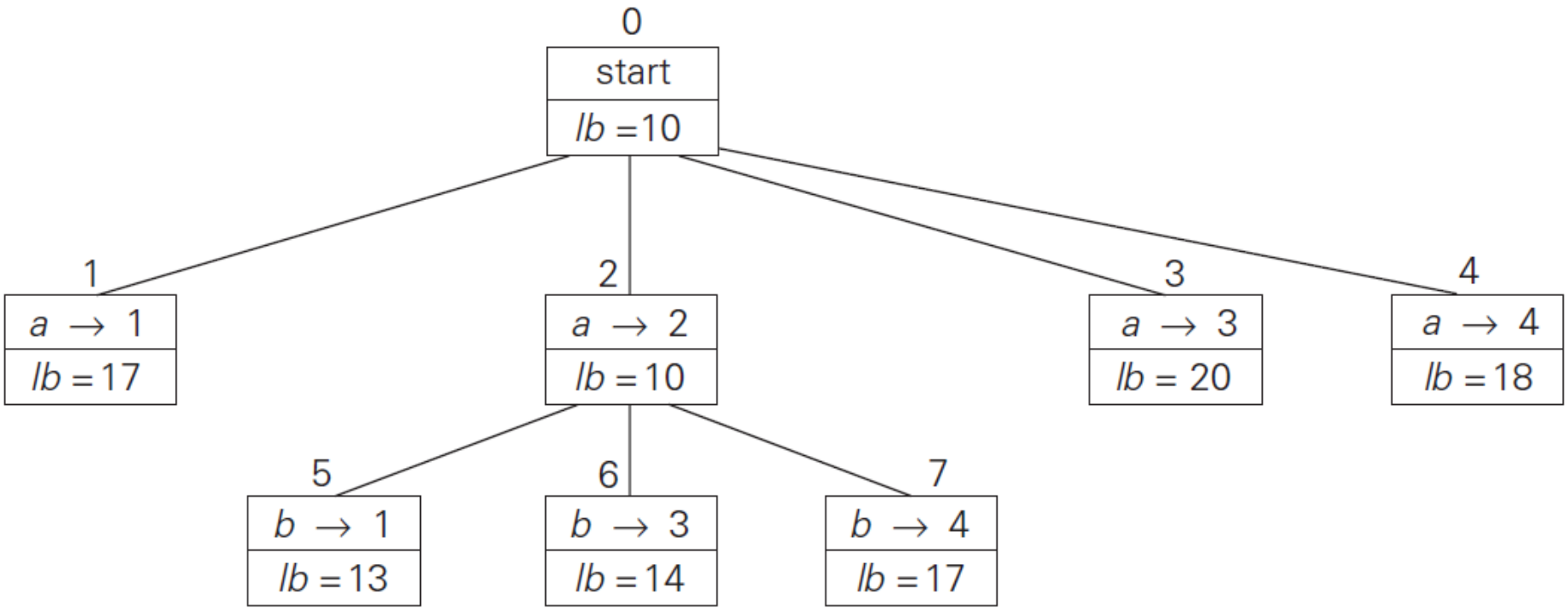
| | Job 1 | Job 2 | Job 3 | Job 4 |
|---|---|---|---|---|
| Person *a* | 9 | 2 | 7 | 8 |
| Person *b* | 6 | 4 | 3 | 7 |
| Person *c* | 5 | 8 | 1 | 8 |
| Person *d* | 7 | 6 | 9 | 4 |

Lower bound: Any solution to this problem will have total cost at least: 2 + 3 + 1 + 4

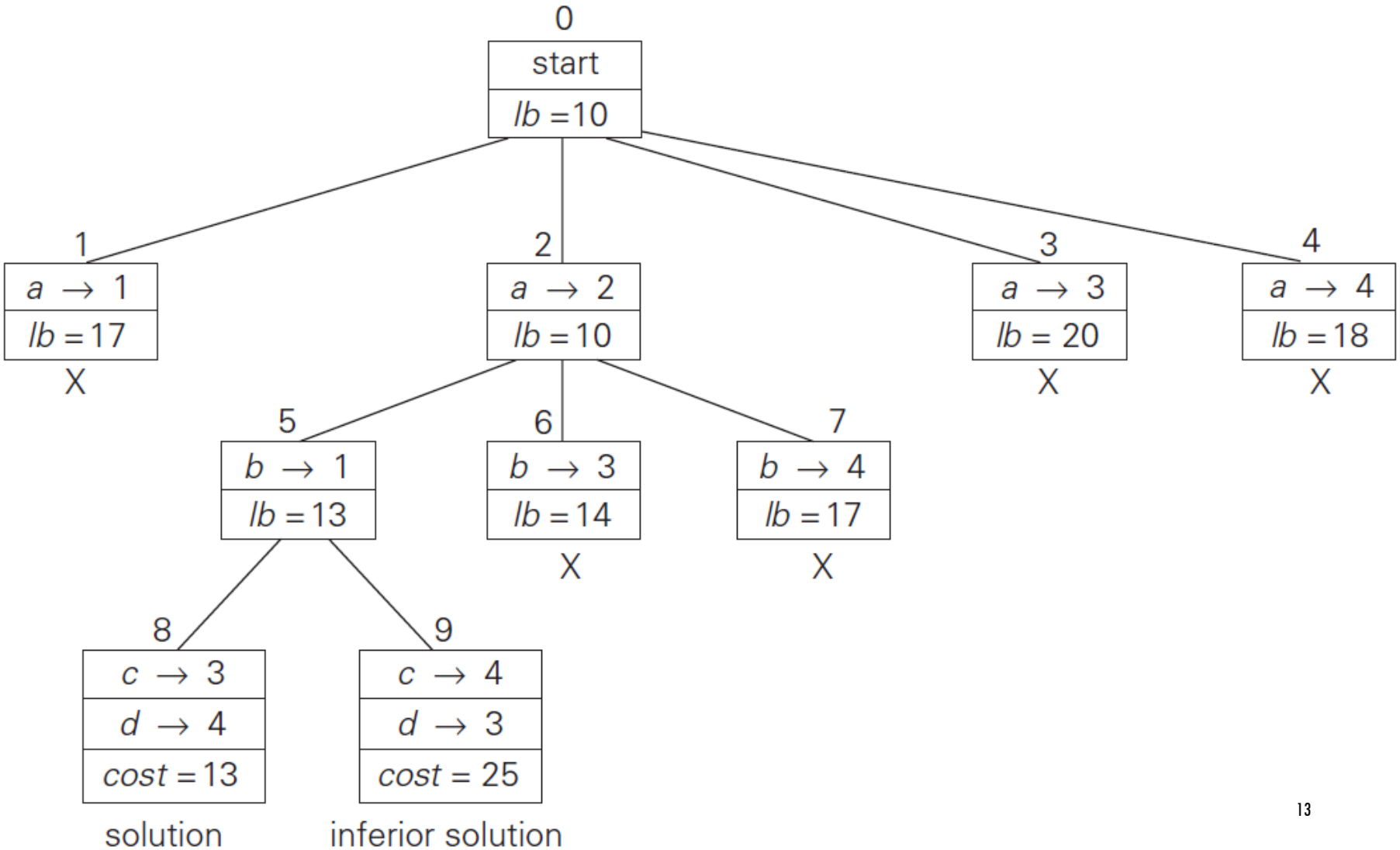
# EXAMPLE: ASSIGNMENT PROBLEM



# EXAMPLE: ASSIGNMENT PROBLEM



# EXAMPLE: ASSIGNMENT PROBLEM



# EXAMPLE: 0-1 KNAPSACK PROBLEM

Given $n$ items of known weights $w_i$ and values $v_i$, $i = 1, 2, \ldots, n$, and a knapsack of capacity $W$, find the most valuable subset of the items that fit in the knapsack.

It is convenient to order the items of a given instance in descending order by their value-to-weight ratios.

$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \frac{v_3}{w_3} \geq \ldots \geq \frac{v_n}{w_n}$$

Each node on the $i$th level of this tree, $0 \leq i \leq n$, represents all the subsets of $n$ items that include a particular selection made from the first $i$ ordered items.

A branch going to the left indicates the inclusion of the next item, and a branch going to the right indicates its exclusion.

# EXAMPLE: 0-1 KNAPSACK PROBLEM

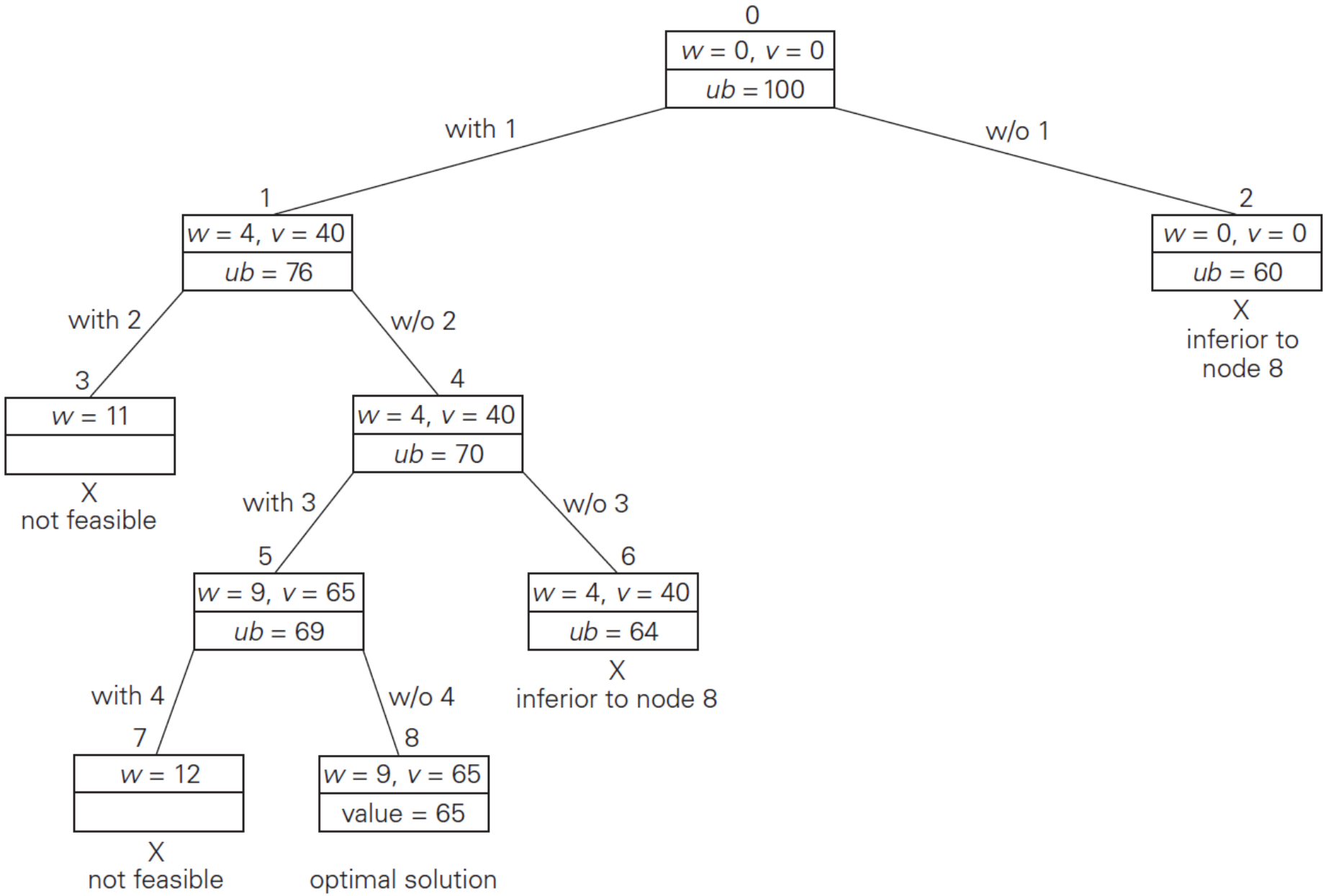
Upper bound $ub$ is to add to $v$, the total value of the items already selected, the product of the remaining capacity of the knapsack $W - w$ and the best per unit payoff among the remaining items, which is $v_{i+1}/w_{i+1}$.

$$ub = v + (W - w)\frac{v_{i+1}}{w_{i+1}}$$

Example

| item | weight | value | $\frac{\text{value}}{\text{weight}}$ |
|---|---|---|---|
| 1 | 4 | \$40 | 10 |
| 2 | 7 | \$42 | 6 |
| 3 | 5 | \$25 | 5 |
| 4 | 3 | \$12 | 4 |

$W = 10$


0
w = 0, v = 0
ub = 100
with 1
w/o 1
1
w = 4, v = 40
ub = 76
2
w = 0, v = 0
ub = 60
X
inferior to node 8
with 2
w/o 2
3
w = 11
X
not feasible
4
w = 4, v = 40
ub = 70
with 3
w/o 3
5
w = 9, v = 65
ub = 69
6
w = 4, v = 40
ub = 64
X
inferior to node 8
with 4
w/o 4
7
w = 12
X
not feasible
8
w = 9, v = 65
value = 65
optimal solution

# EXAMPLE: TRAVELING SALESMAN PROBLEM

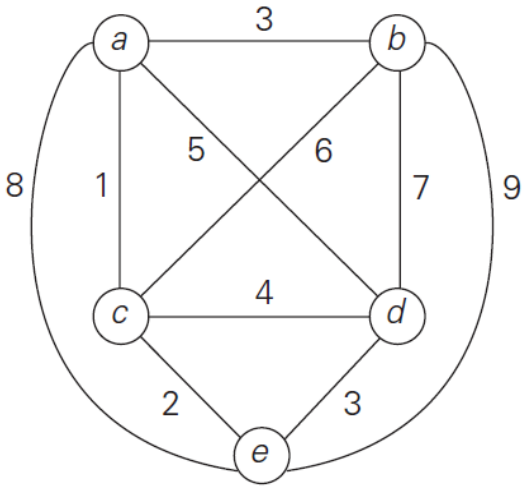
Some possible lower bounds:

1. Finding the smallest element in the intercity distance matrix $D$ and multiplying it by the number of cities $n$.
2. For each city $i$, $1 \leq i \leq n$, find the sum $s_i$ of the distances from city $i$ to the two nearest cities; compute the sum $s$ of these $n$ numbers, divide the result by 2, and, if all the distances are integers, round up the result to the nearest integer:

$$lb = s/2$$

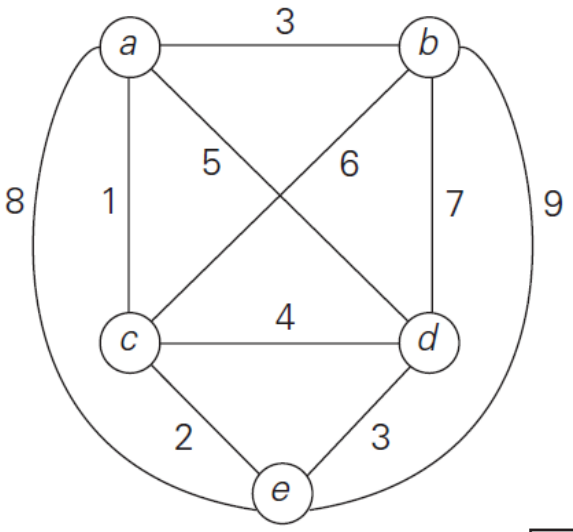
# EXAMPLE: TRAVELING SALESMAN PROBLEM

For example,

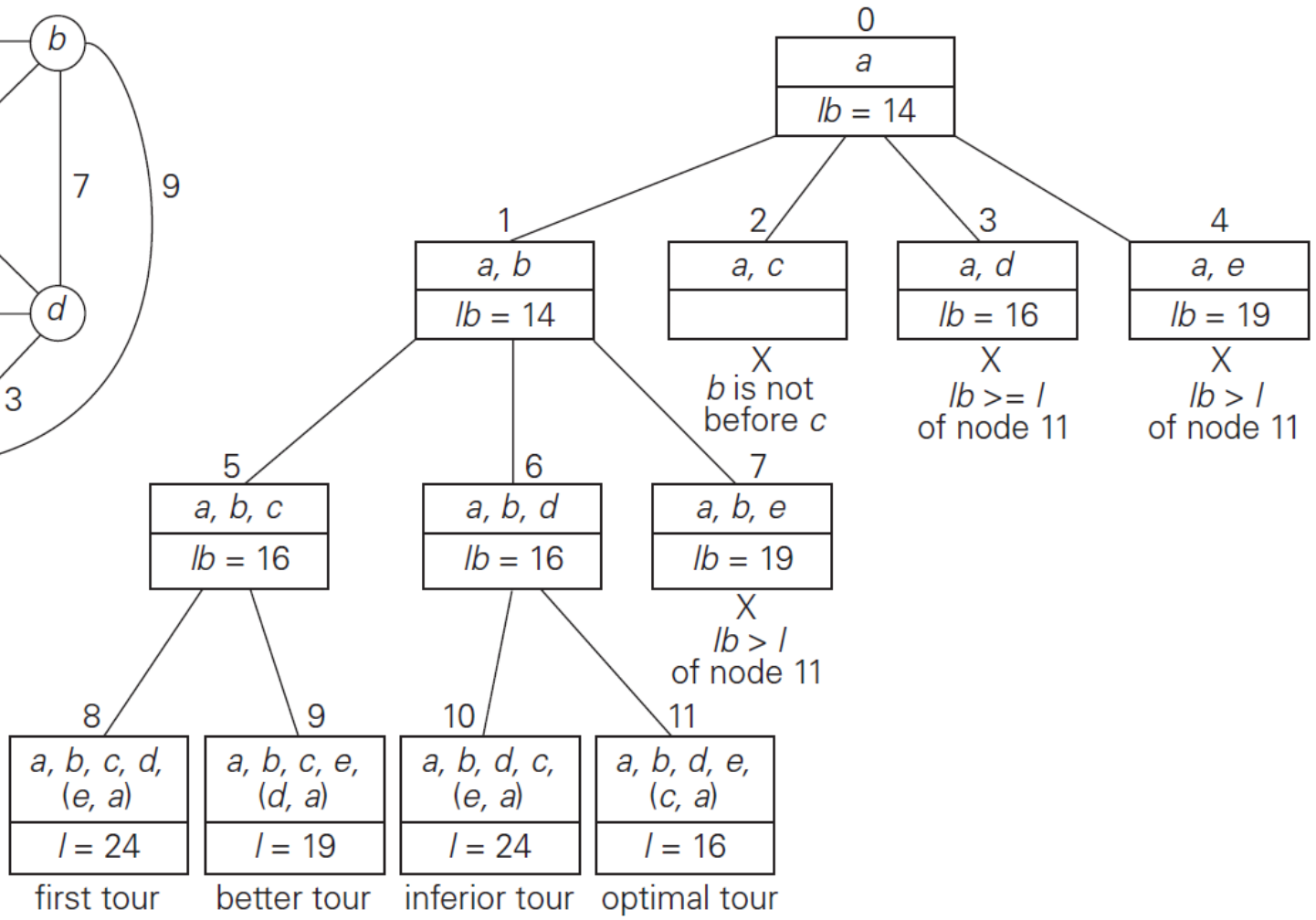



$lb =$.....

For all the Hamiltonian circuits of the graph in that must include edge $(a, d)$, $lb =$.....


a
b
c
d
e
3
5
6
8
1
7
9
4
2
3

(a)


0
a
lb = 14
1
a, b
lb = 14
2
a, c
X
b is not before c
3
a, d
lb = 16
X
lb >= l of node 11
4
a, e
lb = 19
X
lb > l of node 11
5
a, b, c
lb = 16
6
a, b, d
lb = 16
7
a, b, e
lb = 19
X
lb > l of node 11
8
a, b, c, d, (e, a)
l = 24
first tour
9
a, b, c, e, (d, a)
l = 19
better tour
10
a, b, d, c, (e, a)
l = 24
inferior tour
11
a, b, d, e, (c, a)
l = 16
optimal tour

# APPROXIMATION APPROACH

Apply a fast (i.e., a polynomial-time) approximation algorithm to get a solution that is not necessarily optimal but hopefully close to it.

Accuracy measures:

*accuracy ratio* of an approximate solution $s_a$

$$r(s_a) = f(s_a)/f(s^*) \text{ for minimization problems}$$

$$r(s_a) = f(s^*) / f(s_a) \text{ for maximization problems}$$

where $f(s_a)$ and $f(s^*)$ are values of the objective function $f$ for the approximate solution $s_a$ and actual optimal solution $s^*$

A polynomial time approximation algorithm is called $c$ -approximation algorithm if $r(s_a) \leq c$. The smallest $c$ that holds for all instances of the problem is called the performance ratio $R_A$.

# APPROXIMATION ALGORITHMS FOR TSP

Greedy Algorithms:

1. Nearest-Neighbor Algorithm
2. Multifragment-Heuristic Algorithm

Minimum-Spanning-Tree-Based Algorithms

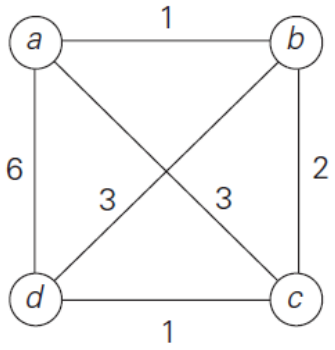
1. Twice-Around-the-Tree Algorithm
2. Christofides Algorithm

Local Search Heuristics (Iterative Improvement Algorithms)

1. 2-Opt Algorithm
2. K-Opt Algorithm
3. Lin-Kernighan Algorithm

# NEAREST-NEIGHBOR ALGORITHM FOR TSP

Starting at some city, always go to the nearest unvisited city, and, after visiting all the cities, return to the starting one




$s_a$: a-b-c-d-a of length 10.

$s^*$: a-b-d-c-a of length 8.

$r(s_a) = \ldots$

Note: Nearest-neighbor tour may depend on the starting city

Accuracy: $R_A = \infty$(unbounded above) – make the length of AD arbitrarily large in the above example

# MULTIFRAGMENT-HEURISTIC ALGORITHM

Stage 1: Sort the edges in nondecreasing order of weights. Initialize the set of tour edges to be constructed to empty set

Stage 2: Add next edge on the sorted list to the tour, skipping those whose addition would've created a vertex of degree 3 or a cycle of length less than $n$. Repeat this step until a tour of length $n$ is obtained

Note: $R_A = \infty$, but this algorithm tends to produce better tours than the nearest-neighbor algorithm

# TWICE-AROUND-THE-TREE ALGORITHM

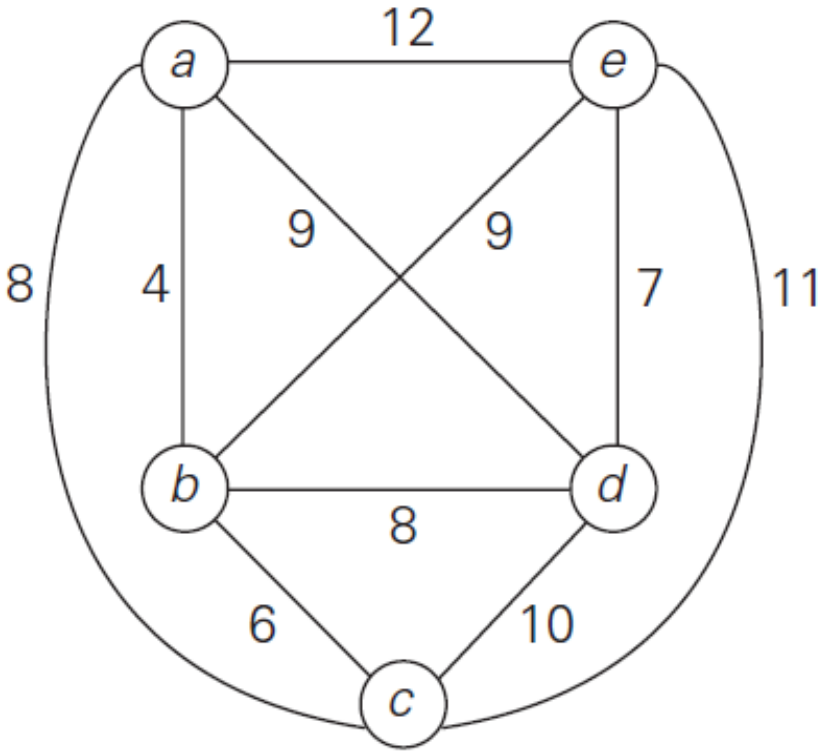
Stage 1: Construct a minimum spanning tree of the graph (e.g., by Prim's or Kruskal's algorithm)

Stage 2: Starting at an arbitrary vertex, create a path that goes twice around the tree and returns to the same vertex

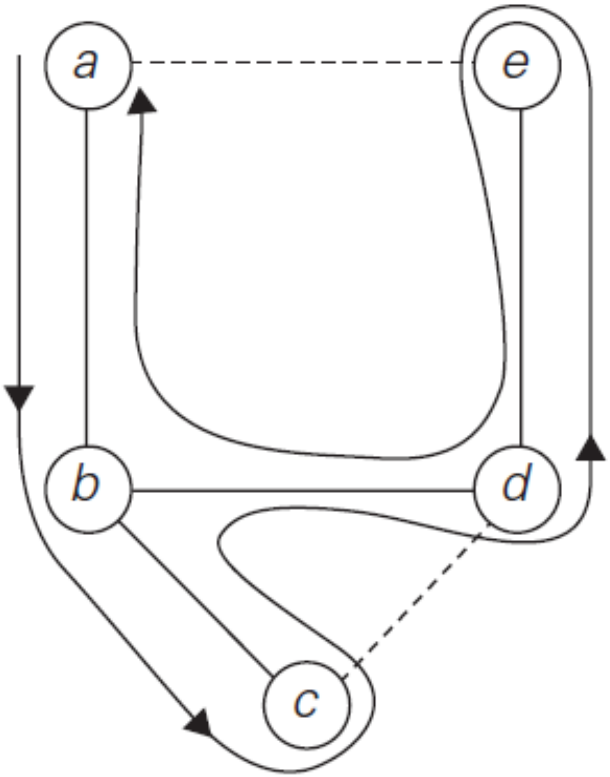
Stage 3: Create a tour from the circuit constructed in Stage 2 by making shortcuts to avoid visiting intermediate vertices more than once

Note: $R_A = \infty$ for general instances, but this algorithm tends to produce better tours than the nearest-neighbor algorithm

# EXAMPLE




Walk: a – b – c – b – d – e – d – b – a




Tour: a – b – c – d – e – a

# CHRISTOFIDES ALGORITHM

Stage 1: Construct a minimum spanning tree of the graph

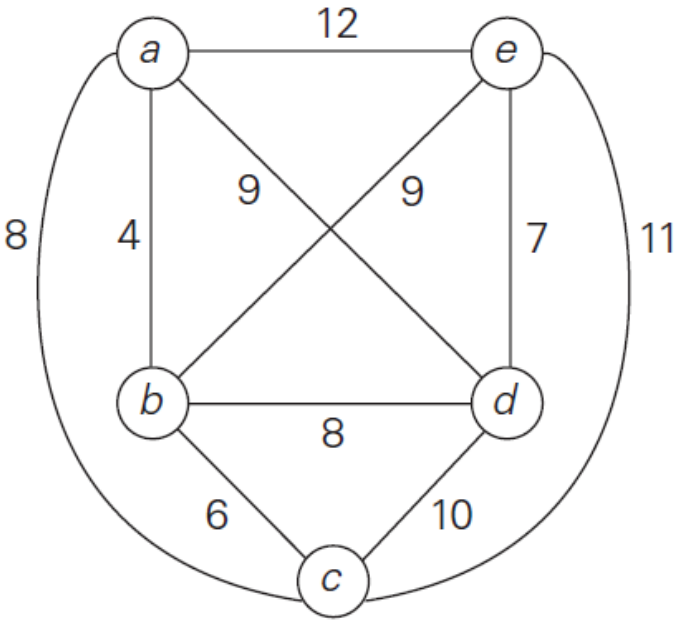
Stage 2: Add edges of a minimum-weight matching of all the odd vertices in the minimum spanning tree

Stage 3: Find an Eulerian circuit of the multigraph obtained in Stage 2

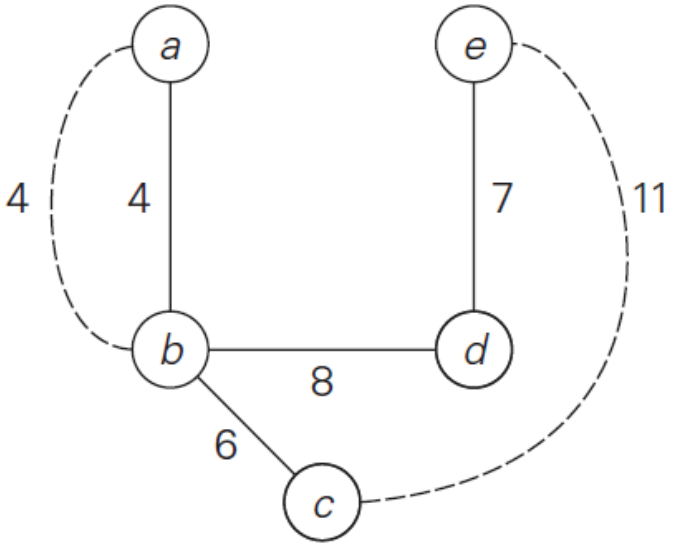
Stage 4: Create a tour from the path constructed in Stage 3 by making shortcuts to avoid visiting intermediate vertices more than once

Note: $R_A = \infty$ for general instances, but it tends to produce better tours than the twice-around-the-minimum-tree algortihm

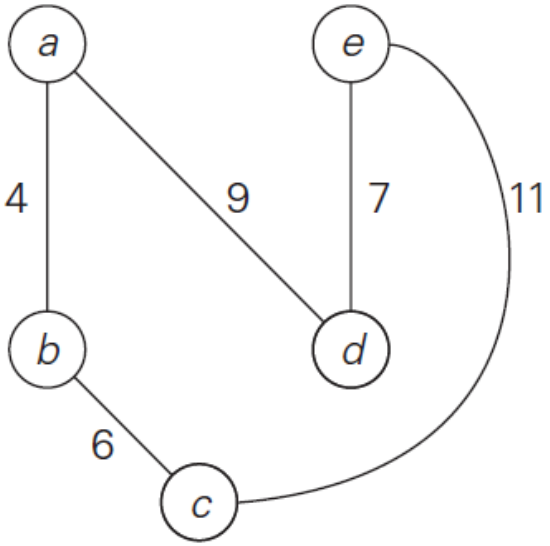
# EXAMPLE




(a)




(b)




(c)

# EUCLIDEAN INSTANCES

Theorem If $P \neq NP$, there exists no approximation algorithm for TSP with a finite performance ratio.

Definition An instance of TSP is called *Euclidean*, if its distances satisfy two conditions:

1. *symmetry* $d[i,j] = d[j,i]$ for any pair of cities $i$ and $j$
2. *triangle inequality* $d[i,j] \leq d[i,k] + d[k,j]$ for any cities $i, j, k$

For Euclidean instances:

$R_A = 0.5(\lceil \log_2 n \rceil + 1)$ for nearest-neighbor and multifragment-heuristic algorithms

$R_A = 2$ for twice-around-the-tree algorithm

$R_A = 1.5$ for Christofides algorithm

# LOCAL SEARCH HEURISTICS FOR TSP

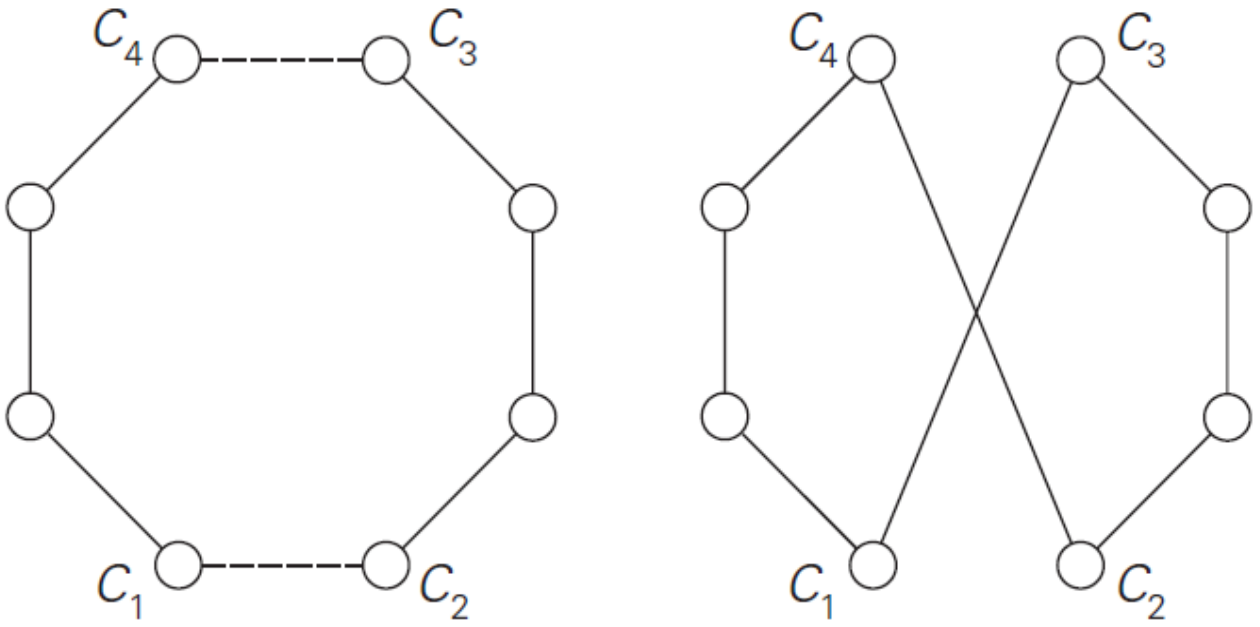
Local search heuristics

Start with some initial tour (e.g., nearest neighbor). On each iteration, explore the current tour's neighborhood by exchanging a few edges in it. If the new tour is shorter, make it the current tour; otherwise consider another edge change. If no change yields a shorter tour, the current tour is returned as the output.

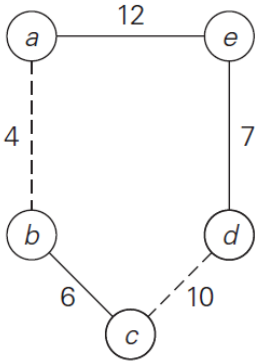
2-opt algorithm

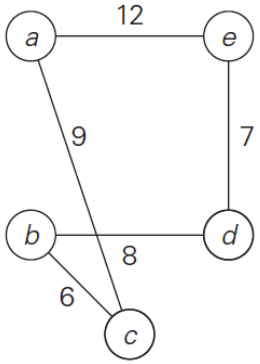
The 2-opt algorithm works by deleting a pair of nonadjacent edges in a tour and reconnecting their endpoints by the different pair of edges to obtain another tour This operation is called the ***2-change***.

# LOCAL SEARCH HEURISTICS FOR TSP

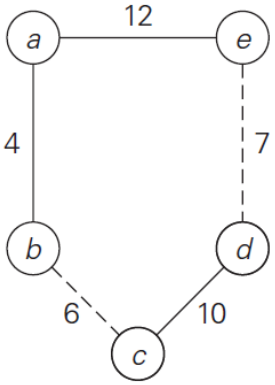


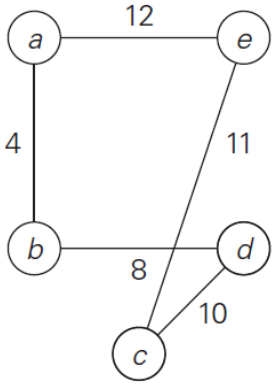
# EXAMPLE



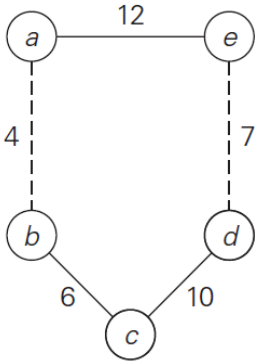



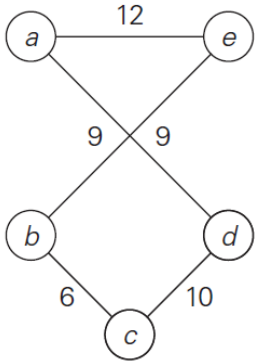

$l = 42 > l_{nn} = 39$



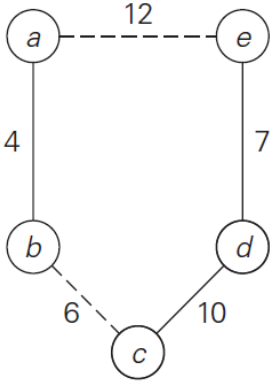



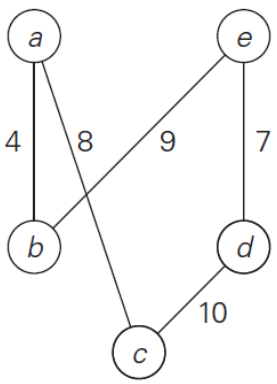

$l = 45 > l_{nn} = 39$




$l = 46 > l_{nn} = 39$



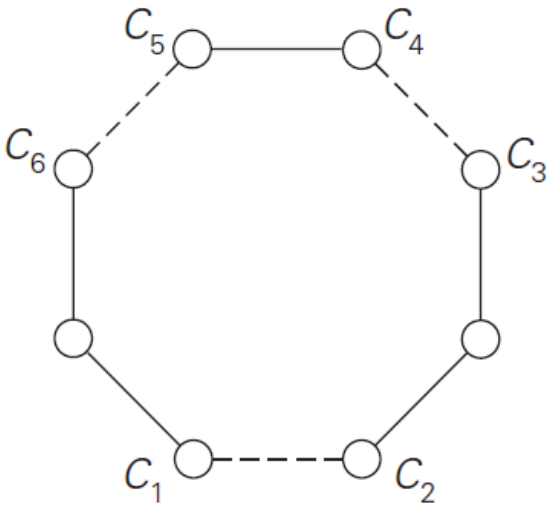



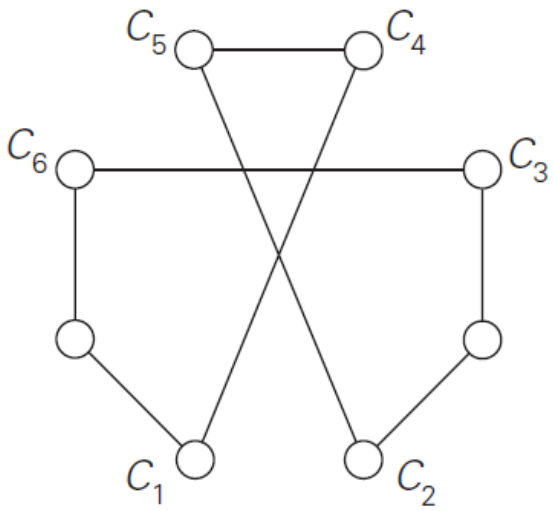

$l = 38 < l_{nn} = 39$
(new tour)

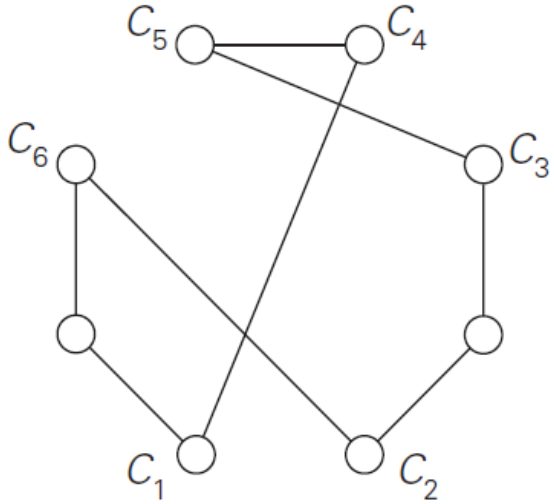
# LOCAL SEARCH HEURISTICS FOR TSP

K-opt algorithm

To generalize the notion of the 2-change, one can consider the ***k-change*** for any $k \geq 2$.



# LOCAL SEARCH HEURISTICS FOR TSP

Lin-Kernighan algorithm

The Lin-Kernighan algorithm which for two decades after its publication in 1973 was considered the best algorithm to obtain high-quality approximations of optimal tours.

The Lin-Kernighan algorithm is a variable-opt algorithm: its move can be viewed as a 3-opt move followed by a sequence of 2-opt moves.

# COMPARISON OF APPROXIMATION ALGORITHMS FOR TSP

Average tour running times for various heuristics on the 10,000-city random uniform Euclidean instances

| Algorithm | Running time (seconds) |
|---|---|
| Nearest neighbor | 0.28 |
| Multifragment | 0.20 |
| Christofides | 1.04 |
| 2-opt | 1.41 |
| 3-opt | 1.50 |
| Lin-Kernighan | 2.06 |

# GREEDY ALGORITHM FOR KNAPSACK PROBLEM

Step 1: Order the items in decreasing order of relative values:

$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \frac{v_3}{w_3} \geq \ldots \geq \frac{v_n}{w_n}$$

Step 2: Select the items in this order skipping those that don't fit into the knapsack

Example: The knapsack's capacity is 10

| item | weight | value | $\frac{\text{value}}{\text{weight}}$ |
|---|---|---|---|
| 1 | 4 | \$40 | 10 |
| 2 | 7 | \$42 | 6 |
| 3 | 5 | \$25 | 5 |
| 4 | 3 | \$12 | 4 |

Example: The knapsack's capacity is $W > 2$

| item | weight | value | value/weight |
|---|---|---|---|
| 1 | 1 | 2 | 2 |
| 2 | $W$ | $W$ | 1 |

# APPROXIMATION SCHEME FOR KNAPSACK PROBLEM

Step 1: Order the items in decreasing order of relative values:

$$\frac{v_1}{w_1} \geq \frac{v_2}{w_2} \geq \frac{v_3}{w_3} \geq \ldots \geq \frac{v_n}{w_n}$$

Step 2: For a given integer parameter $k$, $0 \leq k \leq n$, generate all subsets of $k$ items or less and for each of those that fit the knapsack, add the remaining items in decreasing order of their value to weight ratios

Step 3: Find the most valuable subset among the subsets generated in Step 2 and return it as the algorithm's output

- Accuracy: $f(s^*) \,/\, f(s_a) \leq 1 + 1/k$ for any instance of size $n$
- Time complexity: $O(kn^{k+1})$

# APPROXIMATION SCHEME FOR KNAPSACK PROBLEM

Example: The knapsack's capacity is 10

| item | weight | value | value/weight |
|---|---|---|---|
| 1 | 4 | $40 | 10 |
| 2 | 7 | $42 | 6 |
| 3 | 5 | $25 | 5 |
| 4 | 1 | $ 4 | 4 |

# BIN-PACKING PROBLEM

***Bin-packing problem:*** Given $n$ items whose sizes are positive rational numbers not larger than $1$, put them into the smallest number of bins of size $1$.

*First-Fit (FF)* Algorithm: Consider the items in the order given and place each item in the first available bin with enough room for it; if there are no such bins, start a new one

Example: $n = 4,\ s_1 = 0.4,\ s_2 = 0.2,\ s_3 = 0.6,\ s_4 = 0.7$

Accuracy

Number of extra bins never exceeds optimal by more than 70% (i.e., $R_A = 1.7$)

Empirical average-case behavior is much better. (In one experiment with 128,000 bins, the relative error was found to be no more than 2%.)

# BIN-PACKING PROBLEM

*First-Fit Decreasing (FFD) Algorithm*: Sort the items in decreasing order (i.e., from the largest to the smallest). Then proceed as above by placing an item in the first bin in which it fits and starting a new bin if there are no such bins

Example: $n = 4$, $s_1 = 0.4$, $s_2 = 0.2$, $s_3 = 0.6$, $s_4 = 0.7$

Accuracy

Number of extra bins never exceeds optimal by more than 50% (i.e., $R_A = 1.5$)

Empirical average-case behavior is much better, too

# ACKNOWLEDGEMENTS & REFERENCES


1. A. Levitin, Introduction to the Design & Analysis of Algorithms, 3rd edition, Pearson Education, Inc., 2012.
2. http://www.csl.mtu.edu/cs4321/www/Lectures/Lecture%2028%20-%20Approximation%20Algorithm.htm